

SQL MASTERY

Comparison & Ranking Patterns

A Complete Training Guide for Aspiring SQL Professionals

■ Topics Covered	■ Level	📄 ■ Exercises
ROW_NUMBER · RANK · DENSE_RANK NTILE · LAG · LEAD · Self-Join Subquery Comparison · CASE WHEN	Beginner → Advanced	25+ Hands-On Exercises with Hints & Answers

■ TABLE OF CONTENTS

Chapter 1	Foundations — Why Comparison & Ranking?	3
Chapter 2	ROW_NUMBER() — Unique Sequential Ranking	5
Chapter 3	RANK() and DENSE_RANK() — Handling Ties	8
Chapter 4	NTILE() — Bucketing & Percentile Bands	11
Chapter 5	LAG() and LEAD() — Row-over-Row Comparison	14
Chapter 6	Self-Joins for Comparison	18
Chapter 7	Subquery Comparison Patterns	21
Chapter 8	CASE WHEN for Conditional Ranking	24
Chapter 9	Combining Patterns — Advanced Scenarios	27
Chapter 10	Master Exercises & Cheat Sheet	31

CHAPTER 1

FOUNDATIONS

Why Comparison & Ranking Patterns?

What Are Comparison & Ranking Patterns?

In real-world analytics, raw data alone is rarely enough. Business stakeholders constantly ask questions like: *Who are our top 10 customers? Which product is ranked #1 in each region? How does this month compare to last month?* Answering these questions requires SQL patterns that compare rows against each other and assign meaningful ranks or positions. These patterns form the backbone of analytics, dashboards, and business reporting.

Core Mental Model

Think of a spreadsheet sorted by sales. You mentally scan and say 'row 1 is #1, row 2 is #2...'. SQL ranking functions do this automatically — across any partition, with any sort order, handling ties exactly the way you specify.

The Sample Database We Use Throughout This Guide

All examples use the following three tables. Study them carefully — you'll be writing queries against these tables in every exercise.

employee_id	name	department	salary	hire_date	manager_id
E01	Alice	Engineering	95000	2019-03-15	E10
E02	Bob	Engineering	88000	2020-07-01	E10
E03	Carol	Marketing	72000	2018-11-20	E11
E04	David	Marketing	78000	2021-01-10	E11
E05	Eve	Sales	65000	2022-05-05	E12
E06	Frank	Sales	65000	2022-08-19	E12
E07	Grace	Engineering	88000	2020-07-01	E10
E08	Heidi	HR	55000	2017-06-30	E13
E09	Ivan	HR	60000	2019-09-14	E13
E10	Judy	Engineering	130000	2015-02-01	None

sale_id	employee_id	product	amount	sale_date	region
S001	E01	Laptop	4500	2024-01-05	North
S002	E02	Phone	1200	2024-01-07	South
S003	E05	Tablet	850	2024-01-08	North
S004	E06	Laptop	4500	2024-01-12	West

S005	E01	Phone	1200	2024-02-03	North
S006	E03	Laptop	4500	2024-02-10	South
S007	E05	Phone	1200	2024-02-15	West
S008	E02	Tablet	850	2024-03-01	North
S009	E06	Tablet	850	2024-03-08	South
S010	E01	Laptop	4500	2024-03-20	North

■ NOTE employee_id is the join key between employees and sales. You will frequently JOIN these tables in ranking exercises.

Pattern Overview Map

Pattern	Function/Keyword	Best Used For	Handles Ties?
Sequential Rank	ROW_NUMBER()	Top-N, deduplication	No — always unique
Competitive Rank	RANK()	Sports-style leaderboards	Yes — gaps after ties
Dense Rank	DENSE_RANK()	Category rankings	Yes — no gaps
Bucketing	NTILE(n)	Quartiles, percentiles	Approximate
Row Comparison	LAG() / LEAD()	MoM, WoW deltas	N/A
Self-Comparison	Self-Join	Finding max per group	N/A
Threshold Filter	Subquery	Top-N per partition	N/A
Conditional	CASE WHEN	Custom tier labels	N/A

CHAPTER 2

ROW_NUMBER()

Unique Sequential Ranking

Concept

ROW_NUMBER() assigns a unique integer to every row within a result set. Even if two rows are completely identical, they get different numbers. It is non-deterministic for ties (the order among tied rows depends on the database engine unless you add a tiebreaker).

Syntax

ROW_NUMBER() OVER ([PARTITION BY col1, col2, ...] ORDER BY col [ASC|DESC]) • PARTITION BY — restarts numbering for each group (optional) • ORDER BY — determines which row gets number 1 (required)

Example 1 — Simple Global Rank by Salary

■ Rank all employees by salary (highest = 1)

```
SELECT
    employee_id,
    name,
    department,
    salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

name	department	salary	salary_rank
Judy	Engineering	130,000	1
Alice	Engineering	95,000	2
Bob	Engineering	88,000	3
Grace	Engineering	88,000	4
David	Marketing	78,000	5

■■ IMPORTANT Notice Bob and Grace both earn 88,000 but get ranks 3 and 4 — ROW_NUMBER never repeats. The tie-breaking order between them is arbitrary. To make it deterministic, add employee_id as a secondary sort: ORDER BY salary DESC, employee_id.

Example 2 — Rank Within Department (PARTITION BY)

■ Rank employees by salary within their own department

```
SELECT
    employee_id,
```

```
name,  
department,  
salary,  
ROW_NUMBER() OVER (  
    PARTITION BY department  
    ORDER BY salary DESC  
) AS dept_rank  
FROM employees;
```

name	department	salary	dept_rank
Judy	Engineering	130,000	1
Alice	Engineering	95,000	2
Bob	Engineering	88,000	3
Grace	Engineering	88,000	4
David	Marketing	78,000	1
Carol	Marketing	72,000	2
Ivan	HR	60,000	1
Heidi	HR	55,000	2

■ KEY INSIGHT PARTITION BY restarts the counter for each department. David is rank 1 in Marketing even though globally he is rank 5. This is the most powerful feature of window functions.

Example 3 — Deduplication (Keep Latest Record)

ROW_NUMBER is the standard technique to remove duplicate rows — assign rank 1 to the most recent record per group, then filter WHERE rank = 1.

■ Keep only the most recent sale per employee

```
WITH ranked_sales AS (  
    SELECT  
        employee_id,  
        product,  
        amount,  
        sale_date,  
        ROW_NUMBER() OVER (  
            PARTITION BY employee_id  
            ORDER BY sale_date DESC  
        ) AS rn  
    FROM sales  
)  
SELECT employee_id, product, amount, sale_date
```

```
FROM ranked_sales  
WHERE rn = 1;
```

■ CTE PATTERN You CANNOT use a window function in a WHERE clause directly. Always wrap it in a CTE (WITH ...) or subquery first, then filter on the alias in the outer query.

Exercise 1

★☆☆ Beginner

Write a query that assigns a global row number to all employees ordered by hire_date ascending (oldest hire = 1). Show employee_id, name, hire_date, and the row number.

■ Hint: Use `ROW_NUMBER() OVER (ORDER BY hire_date ASC)`

Your Answer:

Exercise 2

★☆☆ Beginner

Rank employees within each department by hire_date descending (most recently hired = rank 1). Show name, department, hire_date, and dept_hire_rank.

■ Hint: Add `PARTITION BY department` before `ORDER BY`

Your Answer:

Exercise 3

★★☆ Intermediate

Using the sales table, find each employee's first-ever sale (earliest sale_date). Return employee_id, product, amount, sale_date. Use ROW_NUMBER in a CTE and filter rn = 1.

■ Hint: `PARTITION BY employee_id ORDER BY sale_date ASC, then WHERE rn = 1`

Your Answer:

Exercise 4

★★★ Advanced

Find the top-2 highest paid employees in each department. If a department has fewer than 2 employees, still return all of them. Show name, department, salary, and rank.

■ Hint: Use `ROW_NUMBER PARTITION BY department ORDER BY salary DESC, then WHERE rn <= 2`

Your Answer:

CHAPTER 3

RANK() and DENSE_RANK()

Handling Ties Like a Pro

The Tie-Handling Problem

When two employees earn the same salary, what rank should they both get? The answer depends on the business requirement. SQL provides two functions with different tie behavior:

Employee	Salary	ROW_NUMBER	RANK()	DENSE_RANK()
Alice	95,000	1	1	1
Bob	88,000	2	2	2
Grace	88,000	3	2	2
David	78,000	4	4	3
Carol	72,000	5	5	4

■ KEY DIFFERENCE RANK() skips numbers after a tie (after two rank-2s, next is rank 4). DENSE_RANK() never skips — after two rank-2s, next is rank 3. Use RANK() for sports-style leaderboards, DENSE_RANK() for 'what tier is this?'

RANK() Syntax and Example

■ Rank employees by salary — ties share same rank, gaps follow

```
SELECT
    name,
    salary,
    RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees
ORDER BY salary_rank;
```

DENSE_RANK() Syntax and Example

■ Dense rank — no gaps after ties

```
SELECT
    name,
    salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_dense_rank
FROM employees
ORDER BY salary_dense_rank;
```

Real-World Use Case: Find Employees with the Second-Highest Salary

This is an extremely common interview question. Using `DENSE_RANK` ensures that if two employees share the highest salary, the 'second-highest' is the next unique salary level, not a tiebreaker.

■ Second-highest salary using `DENSE_RANK`

```
WITH ranked AS (  
    SELECT  
        name,  
        salary,  
        DENSE_RANK() OVER (ORDER BY salary DESC) AS dr  
    FROM employees  
)  
SELECT name, salary  
FROM ranked  
WHERE dr = 2;
```

PARTITION BY with RANK — Per-Department Champion

■ Find the highest earner in each department

```
WITH dept_ranked AS (  
    SELECT  
        name,  
        department,  
        salary,  
        RANK() OVER (  
            PARTITION BY department  
            ORDER BY salary DESC  
        ) AS dept_salary_rank  
    FROM employees  
)  
SELECT name, department, salary  
FROM dept_ranked  
WHERE dept_salary_rank = 1;
```

■ PRO TIP Filtering `WHERE dept_salary_rank = 1` can return multiple rows per department if there's a tie at the top — which is usually the CORRECT business behavior. If you always want exactly ONE winner per department, use `ROW_NUMBER` instead.

Exercise 5

★ ■ ■ Beginner

Write a query using `RANK()` to rank all employees by salary descending. Show name, salary, and rank. What ranks are skipped if there are ties?

■ *Hint: Compare your output with the table shown in this chapter*

Your Answer:

Exercise 6

★ ■ ■ Beginner

Write a query using `DENSE_RANK()` to rank employees by salary within each department. Show name, department, salary, and `dense_rank`.

■ *Hint: PARTITION BY department ORDER BY salary DESC*

Your Answer:

Exercise 7

★★ ■ Intermediate

Find all employees who have the third-highest salary company-wide. Use `DENSE_RANK` so that ties are respected correctly.

■ *Hint: CTE with DENSE_RANK, then filter WHERE dr = 3*

Your Answer:

Exercise 8

★★★ Advanced

For each region in the sales table, find the employee(s) who had the highest total sales amount. Return region, employee_id, total_amount, and rank. Use `GROUP BY + SUM` first, then `RANK`.

■ *Hint: First GROUP BY region, employee_id to get totals, then RANK OVER (PARTITION BY region ORDER BY total_amount DESC)*

Your Answer:

CHAPTER 4

NTILE()

Bucketing, Quartiles & Percentile Bands

What is NTILE?

NTILE(n) divides rows into n roughly equal buckets and assigns each row a bucket number (1 through n). It is perfect for creating quartiles, quintiles, deciles, or any percentile-based grouping. When rows don't divide evenly, earlier buckets get the extra row.

■ Divide employees into 4 salary quartiles

```
SELECT
    name,
    department,
    salary,
    NTILE(4) OVER (ORDER BY salary DESC) AS quartile
FROM employees
ORDER BY quartile, salary DESC;
```

name	salary	quartile	interpretation
Judy	130,000	1	Top 25% earners
Alice	95,000	1	Top 25% earners
Bob	88,000	2	75th-50th percentile
Grace	88,000	2	75th-50th percentile
David	78,000	3	50th-25th percentile
Carol	72,000	3	50th-25th percentile
Ivan	60,000	4	Bottom 25% earners
Eve	65,000	4	Bottom 25% earners

NTILE with PARTITION BY — Quartiles per Department

■ Salary quartile within each department

```
SELECT
    name,
    department,
    salary,
    NTILE(4) OVER (
        PARTITION BY department
```

```
ORDER BY salary DESC

) AS dept_quartile

FROM employees;
```

■ ■ CAUTION If a department has fewer than *n* employees, some buckets will be empty. For example, Engineering has 4 employees with `NTILE(4)` — each person gets their own quartile. HR has only 2 employees with `NTILE(4)` — only buckets 1 and 2 will be used.

Business Use Case: Customer Segmentation

One of the most common uses of `NTILE` is RFM analysis — splitting customers into spend-based tiers to drive marketing decisions:

■ ■ Label employees by salary tier using `NTILE` + `CASE`

```
SELECT

    name,

    salary,

    NTILE(3) OVER (ORDER BY salary DESC) AS tier_num,

    CASE NTILE(3) OVER (ORDER BY salary DESC)

        WHEN 1 THEN 'High Earner'

        WHEN 2 THEN 'Mid Earner'

        WHEN 3 THEN 'Entry Level'

    END AS salary_tier

FROM employees;
```

Exercise 9

★ ■ ■ Beginner

Split all employees into 5 equal groups (quintiles) based on salary descending. Show name, salary, and quintile number.

■ *Hint: `NTILE(5) OVER (ORDER BY salary DESC)`*

Your Answer:

Exercise 10

★★ ■ Intermediate

Using the sales table, calculate total sales per employee. Then divide employees into 3 tiers (Top, Mid, Lower) based on total sales. Show employee_id, total_sales, and tier_label.

■ *Hint: `GROUP BY + SUM` first, then `NTILE(3)`, then `CASE` for labels*

Your Answer:

Exercise 11

★★★ Advanced

Within each department, rank employees into salary quartiles. Then find only the employees in the TOP quartile (quartile=1) of their department. Return name, department, salary.

■ *Hint: CTE with NTILE(4) PARTITION BY department, then WHERE dept_quartile = 1*

Your Answer:

CHAPTER 5

LAG() and LEAD()

Row-over-Row Comparison

The Core Idea

LAG() and LEAD() let you look at a previous or next row's value without a self-join. They are the standard tool for period-over-period comparisons: month-over-month sales, day-over-day price changes, session time gaps, and more.

Syntax

LAG(column, offset, default) OVER (PARTITION BY ... ORDER BY ...) LEAD(column, offset, default) OVER (PARTITION BY ... ORDER BY ...) • column — the column to peek at • offset — how many rows back (LAG) or forward (LEAD), default = 1 • default — value if no row exists (e.g., first row has no previous)

Example: Month-over-Month Sales Comparison

■ Compare each employee's current sale to their previous sale

```
SELECT
    employee_id,
    sale_date,
    amount,
    LAG(amount, 1, 0) OVER (
        PARTITION BY employee_id
        ORDER BY sale_date
    ) AS prev_sale_amount,
    amount - LAG(amount, 1, 0) OVER (
        PARTITION BY employee_id
        ORDER BY sale_date
    ) AS change_vs_prev
FROM sales
ORDER BY employee_id, sale_date;
```

Example: Monthly Sales Trend with LEAD

■ Look at next month's sale from the current row

```
SELECT
    employee_id,
    sale_date,
    amount,
```

```
    LEAD(amount, 1, NULL) OVER (
        PARTITION BY employee_id
        ORDER BY sale_date
    ) AS next_sale_amount
FROM sales
ORDER BY employee_id, sale_date;
```

Example: Percentage Change

■ Calculate % change from previous sale

```
WITH lagged AS (
    SELECT
        employee_id,
        sale_date,
        amount,
        LAG(amount) OVER (PARTITION BY employee_id ORDER BY sale_date) AS prev_amount
    FROM sales
)
SELECT
    employee_id,
    sale_date,
    amount,
    prev_amount,
    ROUND(100.0 * (amount - prev_amount) / prev_amount, 2) AS pct_change
FROM lagged
WHERE prev_amount IS NOT NULL;
```

■ PRO TIP Always CAST to DECIMAL or use 100.0 * (not 100 *) when computing percentages in SQL to avoid integer division giving 0 unexpectedly.

Example: Consecutive Day Detection (Gap Analysis)

■ Find employees who made sales on consecutive days

```
WITH day_gaps AS (
    SELECT
        employee_id,
        sale_date,
        LAG(sale_date) OVER (
            PARTITION BY employee_id ORDER BY sale_date
        ) AS prev_date,
```

```
sale_date - LAG(sale_date) OVER (
    PARTITION BY employee_id ORDER BY sale_date
) AS days_gap
FROM sales
)
SELECT employee_id, sale_date, prev_date, days_gap
FROM day_gaps
WHERE days_gap = 1;
```

Exercise 12

★★■ Beginner

Using the sales table, for each row show the employee_id, sale_date, amount, and the amount of the PREVIOUS sale by that same employee. Use LAG with a default of 0.

■ *Hint: LAG(amount, 1, 0) OVER (PARTITION BY employee_id ORDER BY sale_date)*

Your Answer:

Exercise 13

★★■ Intermediate

Calculate the running difference between an employee's consecutive sales amounts. Show employee_id, sale_date, amount, prev_amount, and difference. Exclude the first sale (where prev is NULL).

■ *Hint: CTE with LAG, then WHERE prev_amount IS NOT NULL in outer query*

Your Answer:

Exercise 14

★★■ Intermediate

Using LEAD(), for each sale show the next sale date by the same employee. Label the column 'next_sale_date'. If there is no next sale, show NULL.

■ *Hint: LEAD(sale_date) OVER (PARTITION BY employee_id ORDER BY sale_date)*

Your Answer:

Exercise 15

★★★ Advanced

Compute the month-over-month percentage change in total sales amount across ALL employees combined. Group by year-month, compute monthly totals, then use LAG to compare. Show year_month, monthly_total, prev_month_total, pct_change.

■ *Hint: GROUP BY to get monthly totals, then LAG over the monthly totals ordered by year_month*

Your Answer:

CHAPTER 6

SELF-JOINS

Comparing Rows Within the Same Table

What is a Self-Join?

A self-join joins a table to itself using two different aliases. It is used when you need to compare one row against other rows in the same table — for example, finding employees who earn more than their manager, or products cheaper than the average in their category.

Mental Model

Imagine photocopying the employees table twice and calling them 'e1' and 'e2'. Now you can compare any column in e1 against the same column in e2 with full flexibility. The JOIN condition controls which rows get paired together.

Example 1 — Find Employees Earning More Than Their Manager

■ Classic self-join: employee vs manager salary

```
SELECT
    e.name          AS employee_name,
    e.salary         AS employee_salary,
    m.name          AS manager_name,
    m.salary         AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

Example 2 — Find the Maximum Salary Per Department Without Window Functions

■ Self-join approach (pre-window-function era)

```
SELECT DISTINCT e1.department, e1.salary AS max_salary
FROM employees e1
WHERE NOT EXISTS (
    SELECT 1
    FROM employees e2
    WHERE e2.department = e1.department
    AND e2.salary > e1.salary
);
```

■ NOTE The WHERE NOT EXISTS clause reads: 'keep this row only if no other employee in the same department earns MORE than them.' This effectively selects department maxima.

Example 3 — Find All Pairs of Employees in the Same Department

■ Symmetric pair prevention using `e1.employee_id < e2.employee_id`

```
SELECT
    e1.name AS employee_1,
    e2.name AS employee_2,
    e1.department
FROM employees e1
JOIN employees e2
    ON e1.department = e2.department
    AND e1.employee_id < e2.employee_id
ORDER BY e1.department, e1.name;
```

■ TRICK The condition `e1.employee_id < e2.employee_id` prevents duplicate pairs (Alice-Bob AND Bob-Alice) and prevents self-pairs (Alice-Alice). This is a classic combinatorics trick in SQL.

Exercise 16

★ ■ ■ Beginner

Write a self-join query to list each employee alongside their manager's name and salary. Show employee name, employee salary, manager name, and manager salary.

■ Hint: JOIN employees e ON e.manager_id = m.employee_id — use two aliases

Your Answer:

Exercise 17

★★ ■ Intermediate

Find employees who earn LESS than their manager. Show employee name, employee salary, manager name, and manager salary. Order by the salary difference ascending.

■ Hint: WHERE e.salary < m.salary — compute difference in SELECT

Your Answer:

Exercise 18

★★★ Advanced

Using a self-join (no window functions), find the employee with the HIGHEST salary in each department. If there is a tie, return all tied employees.

■ *Hint: LEFT JOIN approach: join e1 against e2 where e2.salary > e1.salary in same dept, then WHERE e2.employee_id IS NULL*

Your Answer:

CHAPTER 7

SUBQUERY COMPARISON PATTERNS

Filtering with Inner Aggregations

Subqueries as Comparison Baselines

A correlated or uncorrelated subquery in the WHERE clause lets you filter rows based on aggregate values — like 'show employees who earn above the company average' or 'show departments whose headcount exceeds the median'. These patterns predate window functions but remain essential.

Pattern 1 — Compare Against Global Aggregate

■ Employees earning above company average salary

```
SELECT name, department, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
)
ORDER BY salary DESC;
```

Pattern 2 — Correlated Subquery (Row-by-Row)

■ Employees earning above their own department average

```
SELECT
    e.name,
    e.department,
    e.salary,
    ROUND(dept_avg.avg_sal, 0) AS dept_avg_salary
FROM employees e
JOIN (
    SELECT department, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY department
) AS dept_avg ON e.department = dept_avg.department
WHERE e.salary > dept_avg.avg_sal
ORDER BY e.department, e.salary DESC;
```

■ TIP Using a derived table JOIN (as above) is almost always faster than a correlated subquery in the WHERE clause, because the inner aggregation runs once instead of once per row.

Pattern 3 — Top-N Per Group Using Subquery

■ Top 2 earners per department — subquery approach

```
SELECT name, department, salary
FROM employees e1
WHERE (
    SELECT COUNT(*)
    FROM employees e2
    WHERE e2.department = e1.department
    AND e2.salary > e1.salary
) < 2
ORDER BY department, salary DESC;
```

■ HOW IT WORKS The subquery counts how many employees in the same department earn MORE than the current row. If fewer than 2 employees earn more, this employee must be in the top 2. This is the classic 'count of better-ranked rows' pattern.

Pattern 4 — IN / NOT IN with Subquery

■ Find employees who have never made a sale

```
SELECT employee_id, name, department
FROM employees
WHERE employee_id NOT IN (
    SELECT DISTINCT employee_id
    FROM sales
)
ORDER BY department;
```

■ NULL TRAP NOT IN behaves unexpectedly when the subquery returns any NULL values — the entire NOT IN returns FALSE for every row. Use NOT EXISTS instead when NULLs are possible: WHERE NOT EXISTS (SELECT 1 FROM sales s WHERE s.employee_id = e.employee_id)

Exercise 19

★ ■ ■ Beginner

Write a query to find all employees whose salary is above the overall company average. Show name, department, salary, and the company average salary in a separate column.

■ Hint: Subquery in WHERE: WHERE salary > (SELECT AVG(salary) FROM employees)

Your Answer:

Exercise 20

★★■ Intermediate

Find employees who earn above the average salary of their own department. Show name, department, salary, and their department's average salary. Round the average to 2 decimal places.

■ *Hint: Use a derived table (subquery in FROM) grouped by department, then JOIN + WHERE e.salary > dept_avg*

Your Answer:

Exercise 21

★★★ Advanced

Using the correlated subquery count pattern, find the top 3 employees by total sales amount. Do not use ROW_NUMBER or RANK — use only subqueries and aggregation.

■ *Hint: Aggregate sales by employee_id, then use: WHERE (SELECT COUNT(*) FROM agg WHERE agg.total > e_total) < 3*

Your Answer:

CHAPTER 8

CASE WHEN

Conditional Ranking & Custom Labels

CASE WHEN as a Ranking Tool

CASE WHEN doesn't rank by itself, but it is essential for converting numeric ranks into meaningful business labels, creating custom tiers based on multiple conditions, and building pivot-style queries.

Pattern 1 — Performance Tier Labeling

■ Convert salary to a performance tier

```
SELECT
    name,
    department,
    salary,
    CASE
        WHEN salary >= 90000 THEN 'Executive'
        WHEN salary >= 75000 THEN 'Senior'
        WHEN salary >= 60000 THEN 'Mid-Level'
        ELSE 'Junior'
    END AS salary_tier
FROM employees
ORDER BY salary DESC;
```

Pattern 2 — Combine CASE WHEN with Window Functions

■ Label top-3 employees per department as 'Elite', rest as 'Standard'

```
WITH ranked AS (
    SELECT
        name,
        department,
        salary,
        DENSE_RANK() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS dept_rank
    FROM employees
)
```

```
SELECT
    name,
    department,
    salary,
    dept_rank,
    CASE WHEN dept_rank <= 3 THEN 'Elite' ELSE 'Standard' END AS category
FROM ranked
ORDER BY department, dept_rank;
```

Pattern 3 — Conditional Aggregation (Pivot)

■ Pivot: total sales amount by region as separate columns

```
SELECT
    employee_id,
    SUM(CASE WHEN region = 'North' THEN amount ELSE 0 END) AS north_sales,
    SUM(CASE WHEN region = 'South' THEN amount ELSE 0 END) AS south_sales,
    SUM(CASE WHEN region = 'West' THEN amount ELSE 0 END) AS west_sales,
    SUM(amount) AS total_sales
FROM sales
GROUP BY employee_id
ORDER BY total_sales DESC;
```

Exercise 22

★ ■ ■ Beginner

Write a query that labels each employee's salary as 'High' ($\geq 85,000$), 'Medium' ($\geq 65,000$), or 'Low' ($< 65,000$). Show name, salary, and tier.

■ Hint: Simple *CASE WHEN salary ≥ 85000 THEN 'High' WHEN salary ≥ 65000 THEN 'Medium' ELSE 'Low'*

Your Answer:

Exercise 23

★★ ■ Intermediate

Combine *NTILE(4)* with *CASE WHEN* to label employees as 'Q1 Top', 'Q2 Upper', 'Q3 Lower', 'Q4 Bottom' based on their salary quartile.

■ Hint: *CTE with NTILE(4), then CASE WHEN quartile = 1 THEN 'Q1 Top' ... etc.*

Your Answer:

Exercise 24

★★★ Advanced

Using conditional aggregation, for each department show the count of employees in each salary tier (High/Medium/Low as defined above) as separate columns: dept, high_count, medium_count, low_count.

■ *Hint: GROUP BY department, SUM(CASE WHEN salary >= 85000 THEN 1 ELSE 0 END) AS high_count*

Your Answer:

CHAPTER 9

COMBINING PATTERNS

Advanced Real-World Scenarios

Scenario 1 — Monthly Sales Ranking with MoM Delta

Task: For each employee, show their monthly total sales, their rank among all employees that month, and the change from the previous month.

■ Combined: GROUP BY + RANK + LAG in a multi-CTE query

```
WITH monthly_totals AS (  
    SELECT  
        employee_id,  
        DATE_TRUNC('month', sale_date) AS sale_month,  
        SUM(amount) AS monthly_total  
    FROM sales  
    GROUP BY employee_id, DATE_TRUNC('month', sale_date)  
,  
ranked AS (  
    SELECT  
        employee_id,  
        sale_month,  
        monthly_total,  
        RANK() OVER (PARTITION BY sale_month ORDER BY monthly_total DESC) AS monthly_rank,  
        LAG(monthly_total) OVER (PARTITION BY employee_id ORDER BY sale_month) AS prev_total  
    FROM monthly_totals  
)  
SELECT  
    r.employee_id,  
    e.name,  
    r.sale_month,  
    r.monthly_total,  
    r.monthly_rank,  
    r.monthly_total - COALESCE(r.prev_total, 0) AS mom_change  
FROM ranked r
```

```
JOIN employees e ON r.employee_id = e.employee_id  
  
ORDER BY r.sale_month, r.monthly_rank;
```

Scenario 2 — Employee Percentile Report

Task: For each employee, show their salary, their NTILE quartile globally, their dense rank within their department, and a combined tier label.

■ Multi-window function report

```
SELECT  
  
    e.name,  
  
    e.department,  
  
    e.salary,  
  
    NTILE(4) OVER (ORDER BY salary DESC) AS global_quartile,  
  
    DENSE_RANK() OVER (  
  
        PARTITION BY department ORDER BY salary DESC  
  
    ) AS dept_dense_rank,  
  
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS global_row_num,  
  
    CASE NTILE(4) OVER (ORDER BY salary DESC)  
  
        WHEN 1 THEN 'Top Earner'  
  
        WHEN 2 THEN 'Above Average'  
  
        WHEN 3 THEN 'Below Average'  
  
        ELSE 'Entry Level'  
  
    END AS compensation_band  
  
FROM employees e  
  
ORDER BY salary DESC;
```

■ **MULTIPLE OVER CLAUSES** You can use multiple different OVER clauses in the same SELECT. Each window function computes independently. You may repeat the same OVER definition multiple times — or define named windows using the WINDOW clause (supported in PostgreSQL, MySQL 8+).

Scenario 3 — Identify Fastest-Growing Employees

Task: Find the employees whose most recent sale amount was the highest percentage increase over their previous sale.

■ LAG + percentage change + RANK combined

```
WITH sale_changes AS (  
  
    SELECT  
  
        employee_id,  
  
        sale_date,  
  
        amount,
```

```
LAG(amount) OVER (PARTITION BY employee_id ORDER BY sale_date) AS prev_amount,
t,
ROW_NUMBER() OVER (PARTITION BY employee_id ORDER BY sale_date DESC) AS recency_rank
FROM sales
),
latest_change AS (
SELECT
    employee_id,
    amount,
    prev_amount,
    ROUND(100.0 * (amount - prev_amount) / prev_amount, 2) AS pct_growth
FROM sale_changes
WHERE recency_rank = 1 AND prev_amount IS NOT NULL
)
SELECT
    lc.employee_id,
    e.name,
    lc.amount AS latest_sale,
    lc.prev_amount,
    lc.pct_growth,
    RANK() OVER (ORDER BY lc.pct_growth DESC) AS growth_rank
FROM latest_change lc
JOIN employees e ON lc.employee_id = e.employee_id
ORDER BY growth_rank;
```

CHAPTER 10

MASTER EXERCISES & CHEAT SHEET

Solidify Everything You've Learned

■ Master Challenge Exercises

Exercise 25

★★★ Advanced

COMPREHENSIVE: Using only the employees and sales tables, write a single query (multi-CTE is fine) that produces this report: For each employee show: name, department, salary, total_sales_amount, salary_rank_in_company (ROW_NUMBER by salary), dept_salary_rank (DENSE_RANK within dept), sales_quartile (NTILE 4 by total sales), performance_label (CASE WHEN: 'Star' if dept_salary_rank=1 AND sales_quartile=1, 'High Performer' if sales_quartile <= 2, else 'Standard'). Order by total_sales_amount DESC.

■ *Hint: CTE 1: aggregate sales; CTE 2: join employees + window functions; CTE 3: CASE WHEN label*

Your Answer:

■ Model Answers for Key Exercises

Exercise 3 Answer

■ Exercise 3: Each employee's first-ever sale

```
WITH ranked AS (
    SELECT employee_id, product, amount, sale_date,
           ROW_NUMBER() OVER (PARTITION BY employee_id ORDER BY sale_date ASC) AS rn
    FROM sales
)
SELECT employee_id, product, amount, sale_date
FROM ranked WHERE rn = 1;
```

Exercise 8 Answer

■ Exercise 8: Highest total sales per region

```
WITH region_totals AS (
    SELECT region, employee_id, SUM(amount) AS total_amount
    FROM sales GROUP BY region, employee_id
),
ranked AS (
```

```
SELECT *, RANK() OVER (PARTITION BY region ORDER BY total_amount DESC) AS rnk
FROM region_totals
)
SELECT region, employee_id, total_amount, rnk FROM ranked WHERE rnk = 1;
```

Exercise 15 Answer

■ Exercise 15: Month-over-Month % change

```
WITH monthly AS (
    SELECT TO_CHAR(sale_date, 'YYYY-MM') AS yr_mo, SUM(amount) AS monthly_total
    FROM sales GROUP BY TO_CHAR(sale_date, 'YYYY-MM')
),
with_lag AS (
    SELECT yr_mo, monthly_total,
           LAG(monthly_total) OVER (ORDER BY yr_mo) AS prev_total
    FROM monthly
)
SELECT yr_mo, monthly_total, prev_total,
       ROUND(100.0*(monthly_total - prev_total)/prev_total, 2) AS pct_change
FROM with_lag WHERE prev_total IS NOT NULL;
```

■ QUICK REFERENCE CHEAT SHEET

Function	Syntax Skeleton	Tie Behavior	Best For
ROW_NUMBER()	ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)	Order by Unique	Dedup, top-N with unique winner
RANK()	RANK() OVER (PARTITION BY ... ORDER BY ...)	Gaps after ties	Sports leaderboards
DENSE_RANK()	DENSE_RANK() OVER (PARTITION BY ... ORDER BY ...)	No Gaps	Category tiers, nth-highest
NTILE(n)	NTILE(n) OVER (ORDER BY ...)	Approximate	Quartiles, percentile bands
LAG(col,n,def)	LAG(col) OVER (PARTITION BY ... ORDER BY ...)	N/A	Previous row, MoM delta
LEAD(col,n,def)	LEAD(col) OVER (PARTITION BY ... ORDER BY ...)	N/A	Next row, forecast diff
Self-Join	FROM t e1 JOIN t e2 ON condition	N/A	Manager compare, pair finding
Subquery	WHERE col > (SELECT AGG FROM ...)	N/A	Above-average, threshold filter
CASE WHEN	CASE WHEN cond THEN val ... END	N/A	Custom labels, pivot columns

■ Common Mistakes & How to Avoid Them

■ Using window function in WHERE directly

Cause: Window functions are computed after WHERE, so filtering on them in WHERE is invalid.

■ Fix: Wrap the window function in a CTE or subquery, then filter in the outer query.

■ NOT IN with NULLs

Cause: NOT IN returns no rows if the subquery contains any NULL, because NULL != anything.

■ Fix: Use NOT EXISTS instead of NOT IN when the subquery might return NULLs.

■ Integer division for percentages

*Cause: 100 * (col1 - col2) / col2 may return 0 if col2 is integer and difference < col2.*

■ Fix: Cast to decimal: 100.0 * (col1 - col2) / col2 or use CAST(col2 AS DECIMAL).

■ ROW_NUMBER tie indeterminism

Cause: ROW_NUMBER on tied values gives arbitrary order — different runs may swap tied rows.

■ Fix: Always add a unique tiebreaker to ORDER BY, such as employee_id.

■ RANK() vs DENSE_RANK() confusion

Cause: Using RANK() when you want the 2nd-highest salary might skip if two rows tie at #1.

■ Fix: For 'nth-highest value' questions always use DENSE_RANK().

You've Got This! ■

Consistency beats intensity. Practice 2-3 exercises daily, and within a month you'll be writing complex window function queries with confidence.

— *Your Senior Developer*